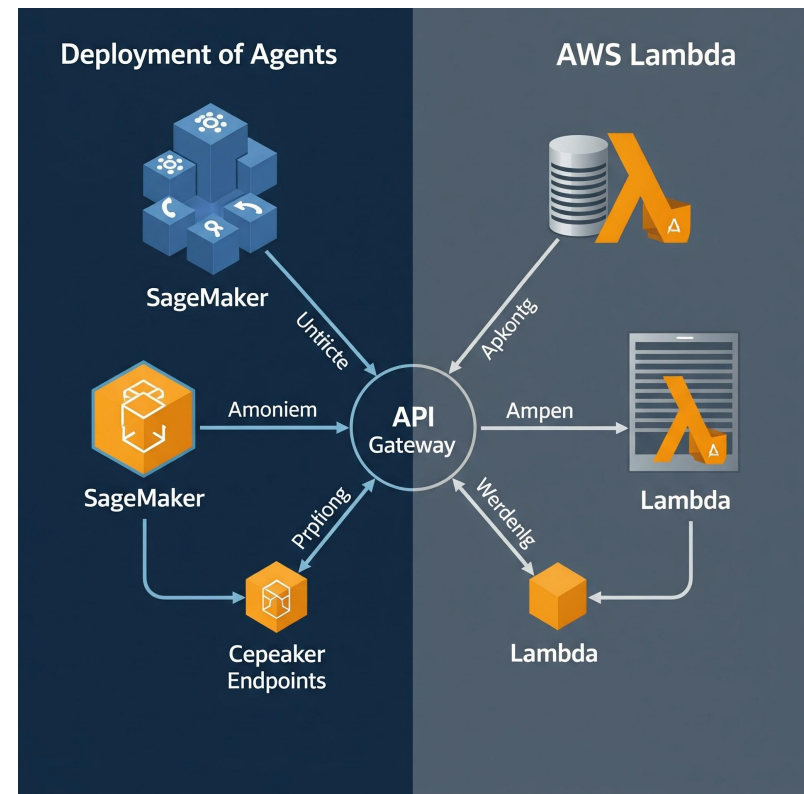


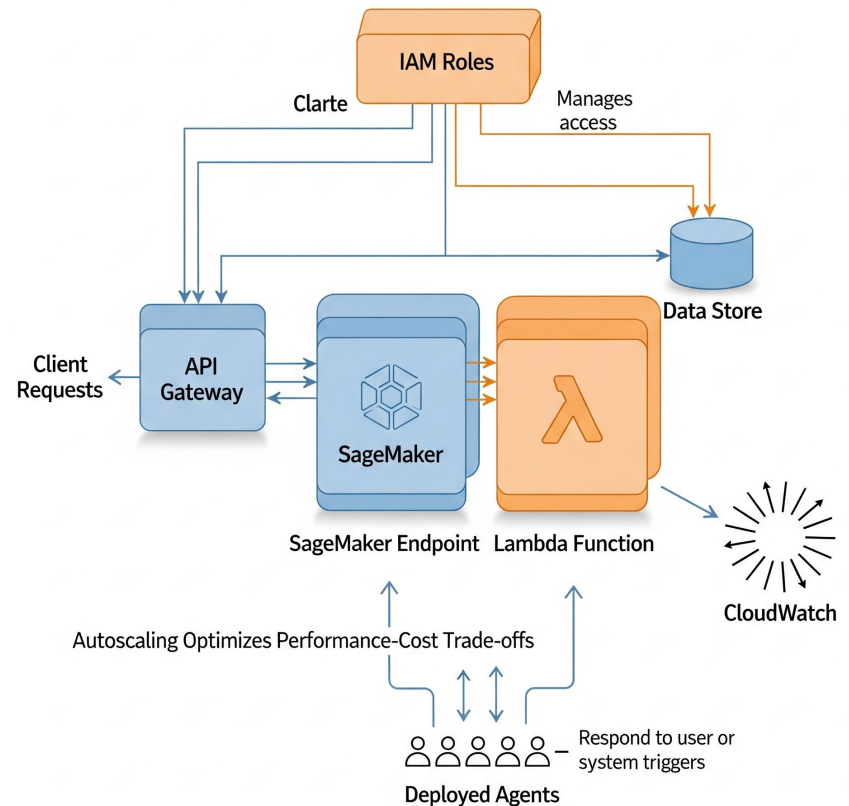
Deploying Agents via SageMaker Endpoints or Lambda

- SageMaker Endpoints provide scalable API hosting for ML/LLM agents.
- AWS Lambda enables lightweight, event-driven deployments.
- Both options integrate seamlessly with API Gateway for REST APIs.
- SageMaker is ideal for long-running or GPU-based agents.
- Lambda fits lightweight, stateless, or event-triggered inference.
- Choice depends on latency, scale, and cost requirements.



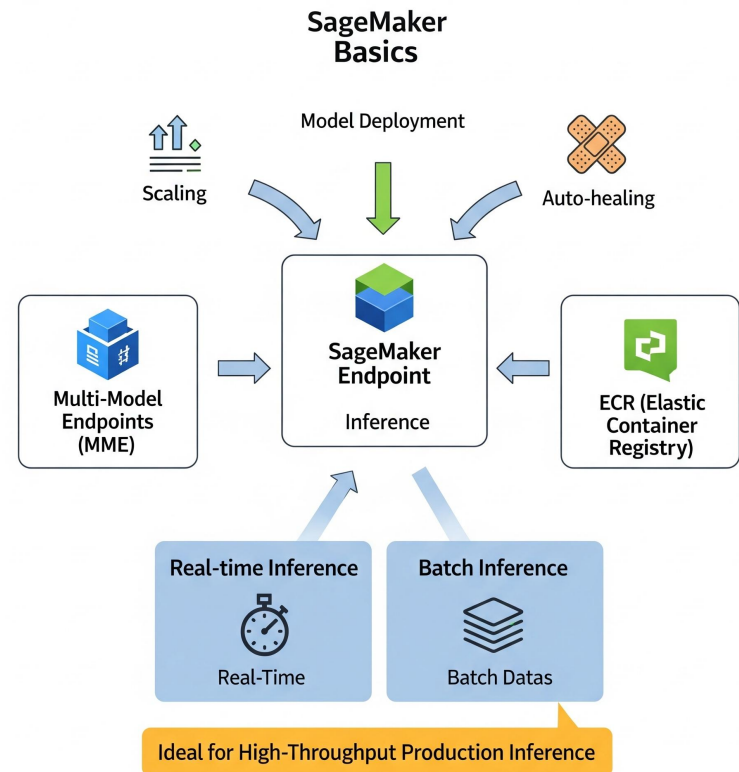
Architecture Overview

- 1. Model hosted as SageMaker Endpoint or Lambda Function.
- 2. API Gateway routes client requests securely.
- 3. IAM roles control access between components.
- 4. CloudWatch monitors usage and performance.
- 5. Autoscaling optimizes performance-cost trade-offs.
- 6. Deployed agents respond to user or system triggers.



SageMaker Endpoint Basics

- Fully managed service for model deployment and inference.
- Supports real-time and batch inference modes.
- Can host multiple models using Multi-Model Endpoints (MME).
- Integrates with ECR for Docker-based model containers.
- Enables custom scaling and auto-healing of endpoints.
- Ideal for high-throughput production inference.



Deploying Model as SageMaker Endpoint

```
from sagemaker.model import Model
model =
Model(model_data='s3://models/agent.tar.
gz',
image_uri='763104351884.dkr.ecr.us-east
-1.amazonaws.com/pytorch:2.0',
role=role)
predictor =
model.deploy(initial_instance_count=1,
instance_type='ml.g4dn.xlarge')
```

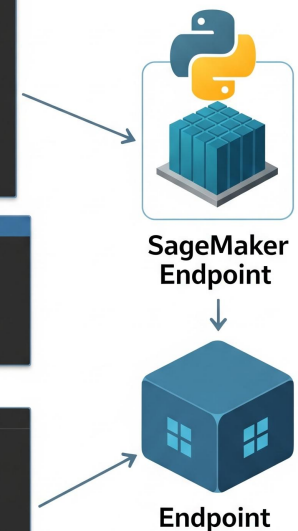
- Provides managed hosting with GPU or CPU support.

Model Definition

```
simple network
//TensorFlow or PyTorch
//
// (TensorFlow PyTorch)
///
}
/imGerayrg)
```

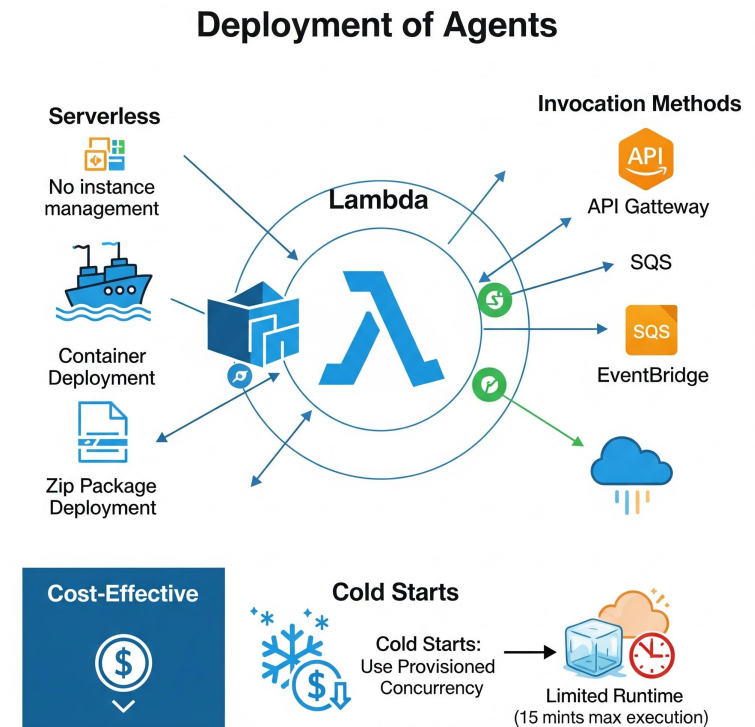
```
SageMaker SageMaker or Model) {
Model
instance_type {
instance.m5.xlarge CPU)
ml.gn44dn.xlarge
entry_point
}
```

```
SageMaker
Model) {
{
SageMaker {
Estimator_type
ml.m5.xlarge CPU
(gdnxlarge GPU
ml.gn4dn.goime GPU)
entry_point
}
```



Lambda for Agent Deployment

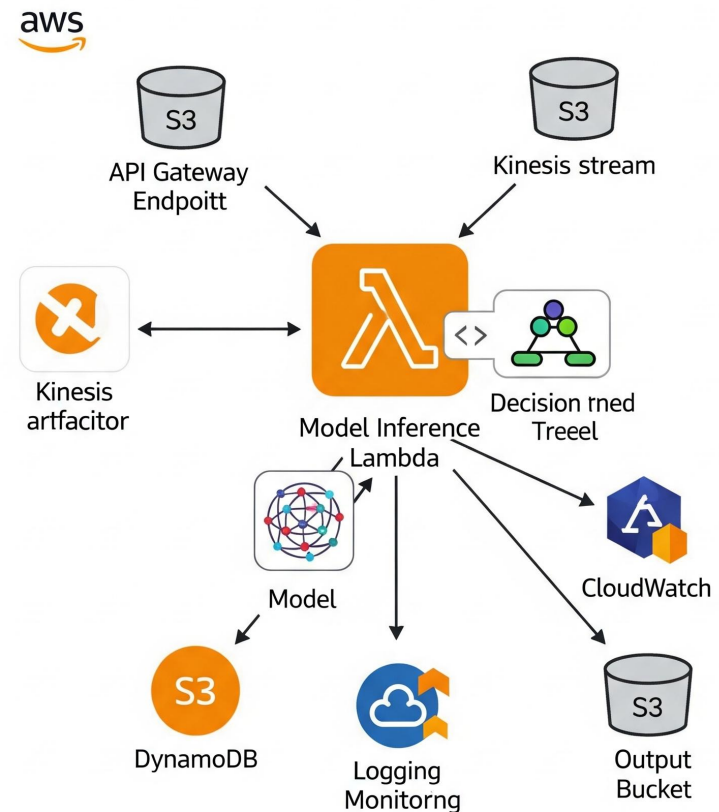
- Lambda is serverless—no instance management required.
- Supports container or zip package-based deployments.
- Invoked on-demand via API Gateway, SQS, or EventBridge.
- Cost-effective for low-traffic or bursty workloads.
- Cold starts may impact latency—use provisioned concurrency.
- Limited runtime (15 mins max execution).



Deploying Model with Lambda

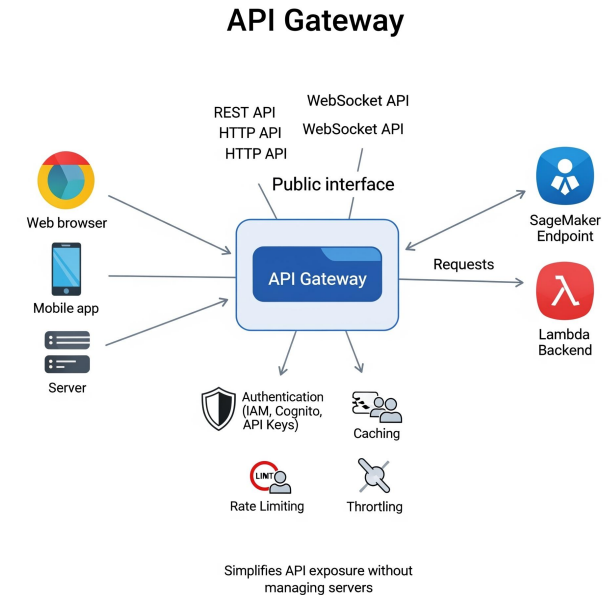
```
import json, boto3
def lambda_handler(event,
context):
    payload =
    json.loads(event['body'])
    result =
    run_agent(payload['query'])
    return {'statusCode': 200,
'body': json.dumps(result)}
```

- Package dependencies and upload to Lambda with AWS CLI.



API Gateway Integration Overview

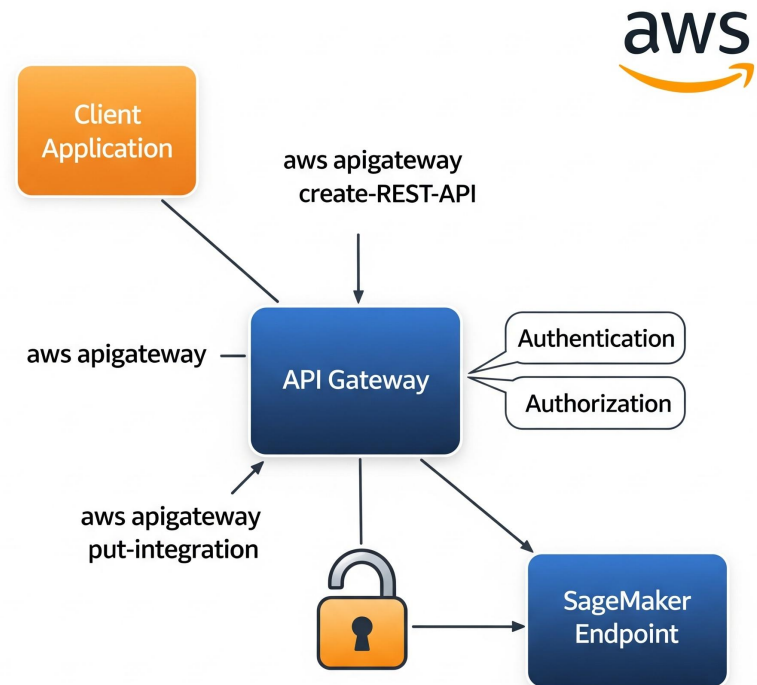
- API Gateway acts as the public interface for agent inference.
- Supports REST, HTTP, and WebSocket APIs.
- Routes requests to SageMaker Endpoint or Lambda backend.
- Handles authentication (IAM, Cognito, API keys).
- Provides rate limiting, caching, and throttling features.
- Simplifies API exposure without managing servers.



Creating API Gateway Integration

```
aws apigateway create-rest-api  
--name 'AgentAPI'  
aws apigateway put-integration  
--rest-api-id API_ID --resource-id  
RESOURCE_ID \  
--http-method POST --type AWS  
--integration-http-method POST \  
--uri  
'arn:aws:apigateway:us-east-1:sag  
emaker:path/endpoints/agent/invoc  
ations'
```

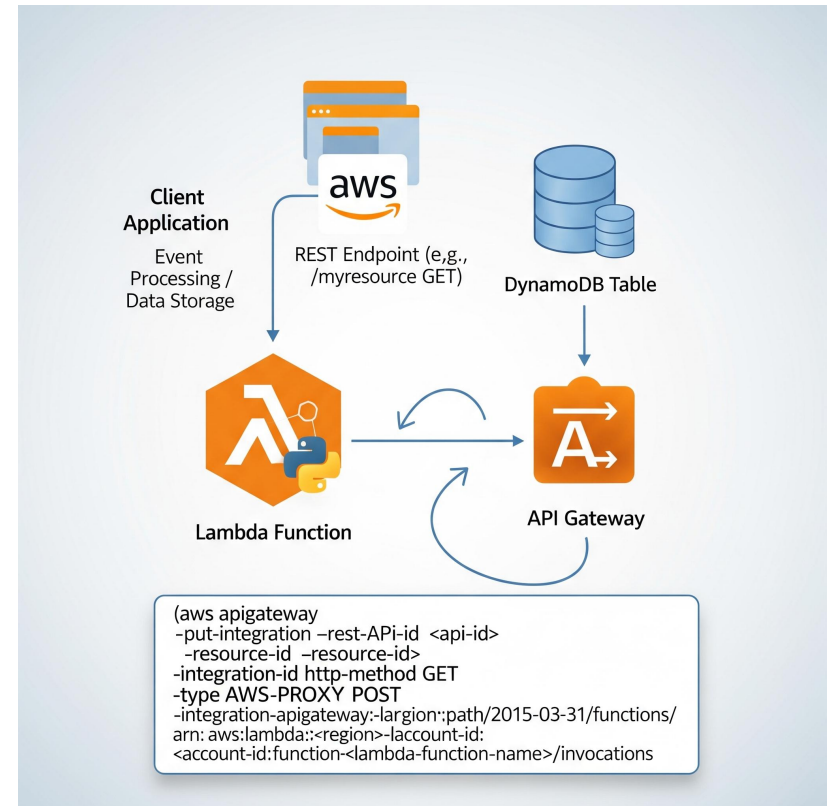
Enables secure connection between client and SageMaker.



Lambda + API Gateway Example

```
aws apigateway put-integration \  
  --rest-api-id API_ID --resource-id  
RESOURCE_ID --http-method POST \  
  --type AWS_PROXY  
--integration-http-method POST \  
  --uri  
'arn:aws:apigateway:us-east-1:lambda:  
path/2015-03-31/functions/LAMBDA_AR  
N/invocations'
```

- Enables event-driven invocation through REST endpoints.



Autoscaling in SageMaker

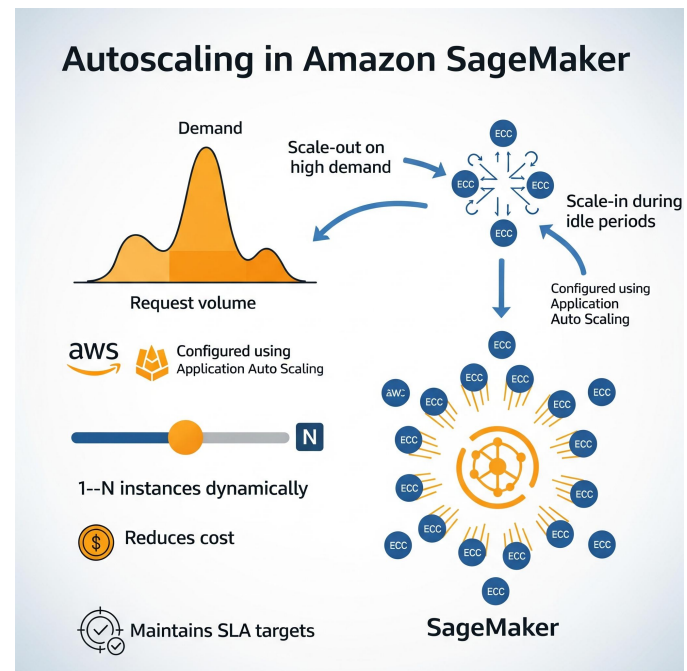
- Automatically adjusts endpoint instance count based on demand.
- Policies configured using Application Auto Scaling service.
- Scale-out on high request volume, scale-in during idle periods.
- Can scale between 1–N instances dynamically.
- Reduces cost without impacting SLA targets.
- Ideal for real-time LLM serving workloads.



Autoscaling Configuration Example

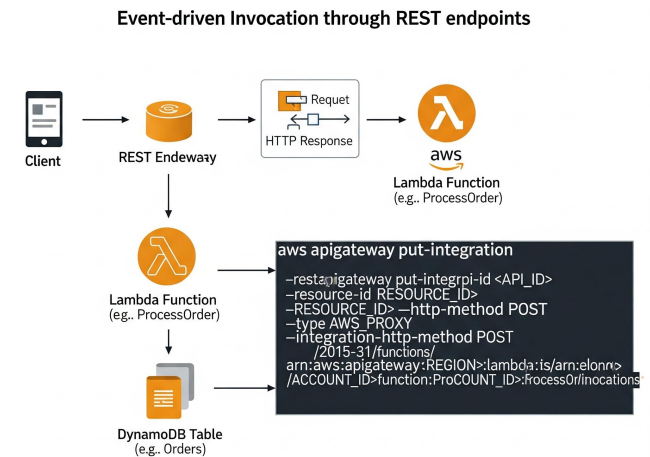
```
aws application-autoscaling
register-scalable-target \
  --service-namespace sagemaker --resource-id
endpoint/agent-endpoint \
  --scalable-dimension
sagemaker:endpoint:DesiredInstanceCount \
  --min-capacity 1 --max-capacity 5
```

- Ensures endpoint scales with traffic while optimizing costs.



Autoscaling for Lambda

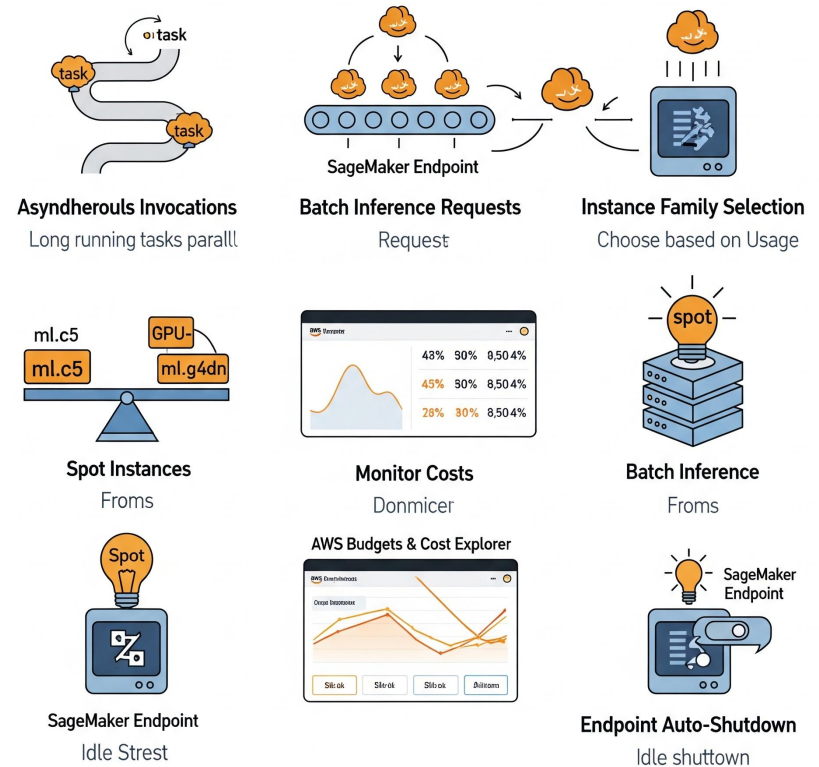
- Lambda scales automatically with incoming request load.
- Each request triggers a concurrent Lambda execution.
- Use reserved concurrency to control max parallel executions.
- Provisioned concurrency minimizes cold start latency.
- Monitor scaling activity using CloudWatch metrics.
- No manual scaling or capacity planning required.



Cost Optimization Strategies

- Use asynchronous invocations for long-running tasks.
- Batch multiple inference requests for SageMaker endpoints.
- Choose instance families (ml.c5 vs ml.g4dn) based on usage.
- Use spot instances for batch inference workloads.
- Monitor costs with AWS Budgets and Cost Explorer.
- Enable endpoint auto-shutdown when idle.

Cost Optimization Strategies for Machine Learning



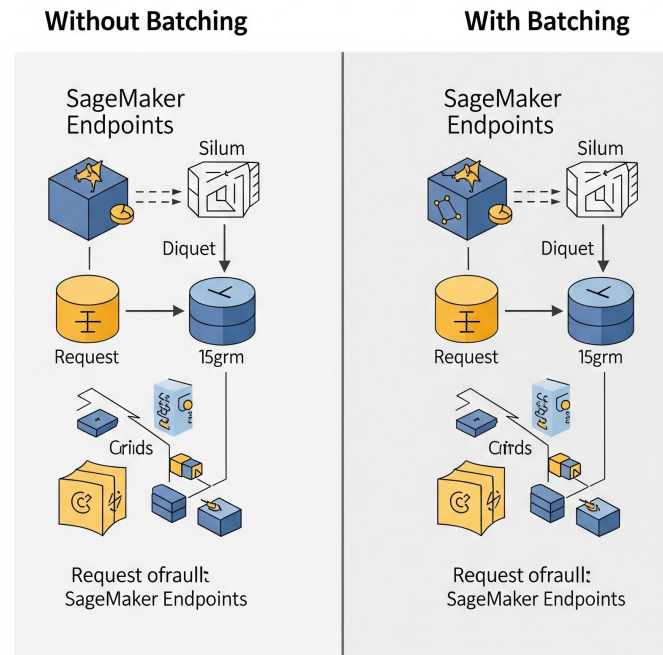
Example: Batching Requests

payloads = [{ 'query': q } for q in queries]

results =

predictor.predict(payloads)

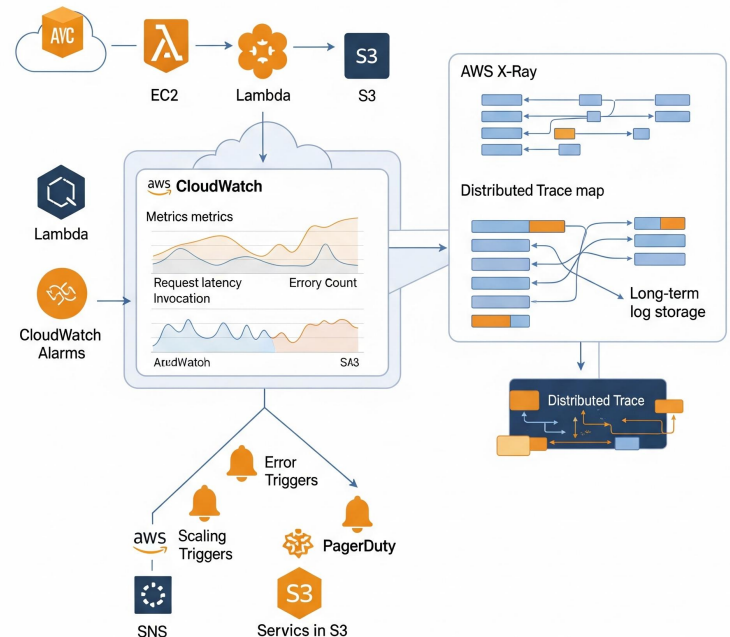
- Batching reduces request overhead and improves throughput.
- Combine multiple queries per network call to minimize latency.
- Helps optimize SageMaker endpoint cost per inference.



Monitoring & Logging

- Enable CloudWatch for logs, metrics, and invocation traces.
- Track request latency, error rate, and invocation count.
- Use CloudWatch alarms for scaling or error triggers.
- Integrate with AWS X-Ray for distributed tracing.
- Store logs for compliance and performance analysis.
- Automate alerts via SNS or PagerDuty integration.

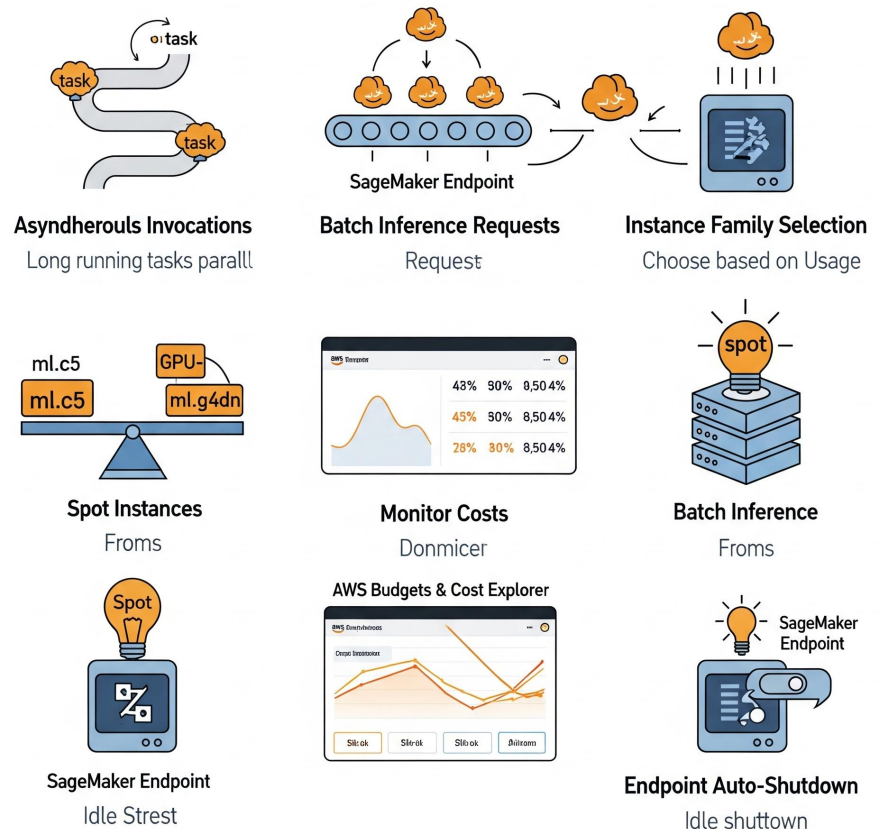
Cloud Monitoring and Logging



Security and IAM Setup

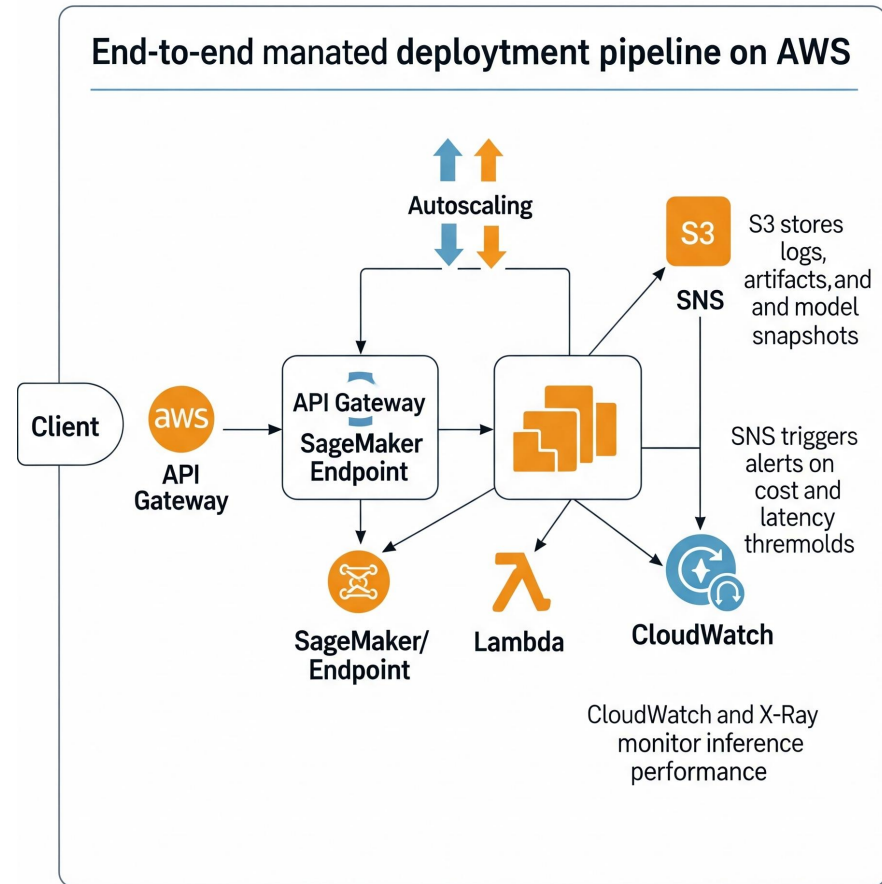
- Grant minimum privileges to Lambda/SageMaker roles.
- Use API Gateway resource policies to restrict access.
- Enable HTTPS via custom domain or ACM certificates.
- Rotate API keys and use Cognito for user authentication.
- Encrypt data at rest (KMS) and in transit (TLS).
- Log access and policy violations via CloudTrail.

Cost Optimization Strategies for Machine Learning



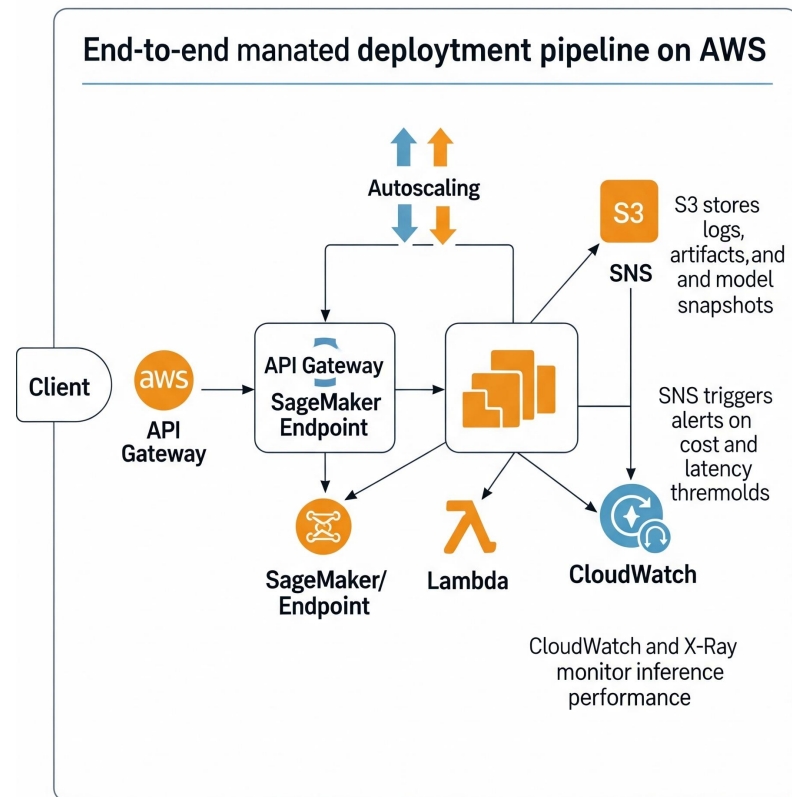
End-to-End Deployment Flow

1. Model built and pushed to S3/ECR.
2. Deploy to SageMaker endpoint or Lambda.
3. Create API Gateway route for client access.
4. Configure autoscaling and cost controls.
5. Enable logging and security policies.
6. Monitor metrics and optimize runtime costs.



Sample Architecture Diagram (Conceptual)

- Client → API Gateway → SageMaker Endpoint/Lambda → CloudWatch.
- Autoscaling adjusts compute automatically based on demand.
- CloudWatch and X-Ray monitor inference performance.
- S3 stores logs, artifacts, and model snapshots.
- SNS triggers alerts on cost or latency thresholds.
- End-to-end managed deployment pipeline on AWS.



Summary and Best Practices

- Choose SageMaker for large or GPU-bound agents; Lambda for light workloads.
- Use API Gateway for secure, managed inference APIs.
- Enable autoscaling to balance performance and cost.
- Batch and async calls reduce cost per inference.
- Apply CloudWatch monitoring and IAM least privilege.
- Continuously optimize endpoints for utilization and latency.

